

11

THE MONTY HALL PROBLEM



As host of the TV game show *Let's Make a Deal*, Monty Hall would show contestants three closed doors and ask them to choose one. Behind one door was a valuable prize; behind the other two were smelly old goats. As soon as the contestant chose a door, Monty would open one of the remaining doors to reveal a goat. The contestant was then given a final choice: switch doors or stay with their initial pick.

In 1990, Marilyn vos Savant, “the world’s smartest woman,” stated in her weekly *Parade* magazine column, “Ask Marilyn,” that the contestant should choose to switch doors. Though her answer was correct, it ignited a firestorm of hate mail, gender profiling, and academic persecution. Many math professors embarrassed themselves in the process, but there was a bright side to the ugly affair. The heated discussion exposed the public to the science of statistics, and an exercise that vos Savant proposed found its way into thousands of classrooms. These manual tests—later duplicated by computers—all vindicated her derided “female logic.”

In this chapter, you’ll use *Monte Carlo simulation (MCS)*, a method for modeling the probability of different outcomes from a range of random inputs, to verify that vos Savant was right. After that, you’ll use `tkinter` to

build a fun graphical interface that addresses her request for schoolchildren to help with the experiment.

Monte Carlo Simulation

Imagine you want to know the probability of rolling a die six times and getting a different face with each roll. If you're a math whiz, you'd probably just compute this using the deterministic equation $6! / 6^6$ or

$$\frac{6}{6} \times \frac{5}{6} \times \frac{4}{6} \times \frac{3}{6} \times \frac{2}{6} \times \frac{1}{6}$$

yielding 0.015. If you're not so mathematically inclined, you could get the same answer with Python and a lot of rolls:

```
>>> from random import randint
>>> trials = 100000
>>> success = 0
>>> for trial in range(trials):
    faces = set()
    for rolls in range(6):
        roll = randint(1, 6)
        faces.add(roll)
    if len(faces) == 6:
        success += 1
>>> print("probability of success = {}".format(success/trials))
probability of success = 0.01528
```

This example uses a `for` loop and `randint` to randomly pick a number between 1 and 6, representing one of the faces on the die, six times in a row. It adds each result to a set named `faces`, which doesn't permit duplicates. The only way for the length of the set to reach 6 is if each roll yields a unique number, which equals a success case. An outer `for` loop performs the six-roll trial 100,000 times. Dividing the number of successes by the number of trials yields the same probability, 0.015, as the deterministic equation.

Monte Carlo simulation uses *repeated random sampling*—in this case, each roll of the die is a random sample—to predict different outcomes under a specified range of conditions. For this example, the range of conditions was one six-faced die, six rolls with no repeats per trial, and 100,000 trials. Of course, MCS is usually applied to more complex problems—those with lots of variables and wide ranges of uncertainty, where the results cannot be easily predicted.

There are multiple types of MCS, but most applications follow these basic steps:

- List the input variables.
- Provide a probability distribution for each variable.
- Start a loop:
 - Randomly select a value from the distributions for each input.
 - Use the values in a deterministic calculation, which is a calculation that will always produce the same output from the same input.
 - Repeat a specified number of times.
- Aggregate the results and generate statistics, such as the average outcome for the calculation.

For the die-roll example, these steps were:

- Input variables = the results of six die rolls.
- Probability distribution for roll = uniform (1/6 for each face).
- Loop:
 - Randomly selected value = die roll (draw from distribution).
 - Calculation = add the six values to a set and, if set length equals 6, add 1 to `success` variable.
 - Repeat = 100,000 times.
- Aggregate: divide `success` variable by 100,000 for probability calculation.

Nassim Taleb, the critically acclaimed author of *The Black Swan* and *Fooled by Randomness*, is a fan of MCS. He posits that our brains are designed to get us out of trouble quickly, rather than handle complicated uncertainty or probability problems. We aren't cut out for highly skewed distributions and nonlinearities, but some people's brains are inherently

more capable of understanding risks using MCS than other methods. In real life, we don't observe probability distributions; we just observe events.

Each MCS run represents a single event, such as whether you run out of money in retirement. For many of us, MCS makes risk real. It helps us understand how bad or good things can be—something we can't always glean from mathematical abstractions. With the insight from MCS, we can prepare to both defend against the downside and exploit the upside.

To support the math behind the Monty Hall problem, you'll use an MCS application like the preceding die-roll example. Then, in **Chapter 12**, you'll use MCS to build a nest-egg simulator to plan your (or your parents') secure retirement.

Project #18: Verify vos Savant

To verify that vos Savant was right, use a Monte Carlo approach and simulate tens of thousands of “games” in order to see how things shake out. This can be a bare-bones program, since the goal is a simple confirmation with no embellishments.

THE OBJECTIVE

Write a simple Python program that uses Monte Carlo simulation to determine the probability of winning the Monty Hall problem by changing the initial pick.

The Strategy

The correct response to the Monty Hall problem is to switch doors after Monty reveals the goat. Statistically, this will double your chances of winning!

Look at **Figure 11-1**. At the start of the game, all the doors are closed, and the odds of a given door hiding the prize are 1 in 3. The user can choose

only one door, which means the odds of the prize being behind one of the other two doors are 2 in 3. After the goat is revealed, the odds remain 2 in 3, but they revert to the remaining door. Remember, Monty knows where the prize is hidden, and he will never reveal *that* door. So the probability of success is 1/3 for staying with your first choice versus 2/3 for switching.

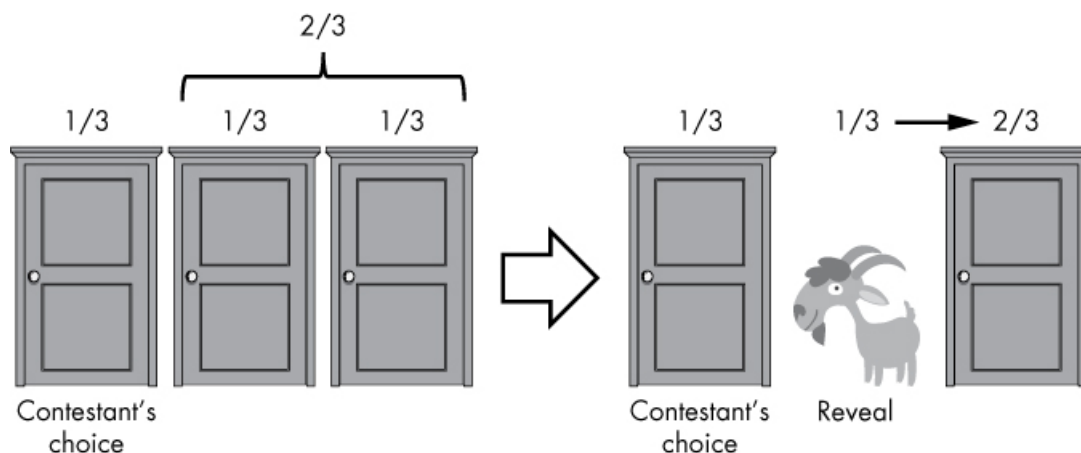


Figure 11-1: Odds of winning the Monty Hall problem before and after a goat is revealed

If you're dubious about the math, you can use MCS to provide corroborating evidence, just as we did with the die-roll example. You simply need to pick a winning door at random, choose a contestant's pick at random, and record how many times the two coincide. Repeat this thousands of times, and you will converge on the deterministic mathematical solution.

The vos Savant Verification Code

The `monty_hall_mcs.py` program described in this section will automate the process of choosing doors and recording the results so that you can run thousands of trials and evaluate them in less than a second. You can download the code from

<https://www.nostarch.com/impracticalpython.com/>.

Getting the Number of Runs Input

Listing 11-1 starts the `monty_hall_mcs.py` program by asking the user how many runs—or games—they want to simulate. You'll also provide the user with a default value. This is a great way to guide a user to a reasonable first response, as well as save them a few keystrokes.

```

❶ import random

❷ def user_prompt(prompt, default=None):
    """Allow use of default values in input."""
    ❸ prompt = '{} [{}]: '.format(prompt, default)
    ❹ response = input(prompt)
    ❺ if not response and default:
        return default
    else:
        return response

    # input number of times to run simulation
❻ num_runs = int(user_prompt("Input number of runs", "20000"))

```

Listing 11-1: Imports modules and defines the `user_prompt()` function

Start by importing the `random` module to run MCS ❶. Next, define a function, called `user_prompt()`, that asks the user to either input the number of games to run or else accept a default value, if provided ❷. This function takes two arguments; the first is the text prompt that tells the user what to do, and the second is the default value, which will start out as `None`. Immediately redefine the `prompt` variable so that it will display with the default value in brackets, per convention ❸. Assign the user's input to a variable named `response` ❹. If the user presses ENTER without providing any input and a default value exists, the `user_prompt()` function will return the default value ❺. Otherwise, the function returns the user's input. Use the function to determine the number of runs to make by assigning the returned value to the `num_runs` variable ❻. Each run will represent a contestant playing the game once.

Running MCS and Displaying the Results

Listing 11-2 picks the winning door and the user's first choice at random, then aggregates and presents the statistics. Interestingly, the user's second choice—to switch doors or not—isn't required to get the correct answer.

If the initial choice is the winning door, the correct answer is to not change doors. Likewise, if the initial choice and the winning door differ, the correct answer is to change doors. There's no reason to model what a contestant might or might not do.

monty_hall_mcs.py, part 2

```
# assign counters for ways to win
❶ first_choice_wins = 0
   pick_change_wins = 0
❷ doors = ['a', 'b', 'c']

# run Monte Carlo
❸ for i in range(num_runs):
    winner = random.choice(doors)
    pick = random.choice(doors)

    ❹ if pick == winner:
        first_choice_wins += 1
    else:
        pick_change_wins += 1

❺ print("Wins with original pick = {}".format(first_choice_wins))
   print("Wins with changed pick = {}".format(pick_change_wins))
   print("Probability of winning with initial guess: {:.2f}"
         .format(first_choice_wins / num_runs))
   print("Probability of winning by switching: {:.2f}"
         .format(pick_change_wins / num_runs))

❻ input("\nPress Enter key to exit.")
```

Listing 11-2: Runs the Monte Carlo simulation and displays the results

Assign two variables to keep track of whether switching or staying put is the winning outcome ❶. Then, create a list to represent the three doors ❷.

MCS starts with a `for` loop that goes through the number of runs ❸. Inside that loop, choose the winning door and the user's first choice from the `doors` list, using `random.choice()`, and assign them to variables.

Since this is a binary system—the user switches or doesn't—you'll only need a conditional that adds to the counters based on the relationship of the `pick` variable to the `winning` variable ❹.

Finish the program by presenting the final results. Display the actual counts, plus the calculated probabilities ❺. Then let the user know the program is finished ❻.

Here's an example output for the default 20,000 runs:

```
Input number of runs [20000]:
Wins with original pick = 6628
Wins with changed pick = 13372
Probability of winning with initial guess: 0.33
Probability of winning by switching: 0.67

Press Enter key to exit.
```

Some people aren't impressed with a computer printout. They need something more convincing, so in the next project, you'll repackage your code in a more hands-on format—one that's complete with doors, prizes, and goats. This will also satisfy Marilyn vos Savant's appeal for schoolchildren to join in and help restore her honor.

Project #19: The Monty Hall Game

The three-door game used in the Monty Hall problem is simple enough for you to build with `tkinter`. You began working with `tkinter` graphics in **Chapter 10**. Now you'll build on this knowledge by adding interactive buttons for the user to click.

Simulate the Monty Hall problem using a GUI built with `tkinter`. Keep track of whether switching doors or staying put results in a win. In addition, update and display these statistics as the game is played.

A Brief Introduction to Object-Oriented Programming

The `tkinter` module was written using *object-oriented programming (OOP)*. OOP is a language model built around data structures, known as *objects*, consisting of *data* and *methods* and the interactions between them—as opposed to the *actions* and *logic* used in procedural programming. Objects are built from *classes*, which are like blueprints for the objects.

OOP is an abstract concept and easier to appreciate when you're writing large, complex programs. It reduces code duplication and makes code easier to update, maintain, and reuse. As a result, most commercial software is now built using OOP.

If you implemented OOP in small programs, like the ones we've written so far, most of them would feel overengineered. In fact, one of my all-time favorite quotes, attributed to British computer scientist Joe Armstrong, concerns this aspect of OOP: “The problem with object-oriented languages is they've got all this implicit environment that they carry around with them. You wanted a banana, but what you got was a gorilla holding the banana and the entire jungle!”

Despite this, the objects produced by OOP lend themselves very well to GUIs and gaming, even for some small projects. Let's look at an example using a *Dungeons and Dragons*–type board game in which players can be different characters, such as dwarves, elves, and wizards. These games use character cards to list important information for each character type. If you let your playing piece represent a dwarf, it inherits the characteristics on the card (see **Figure 11-2**).

Dwarf

You are the dwarf. You are a small but tough fighter with your broadax. You are skilled at finding and disarming traps.

Attack
Throw 3 Combat Dice

Defend
Throw 4 Combat Dice

Move
Throw 2 Standard Dice

Body Points
5



Figure 11-2: A dwarf character card from a role-playing board game

Listings 11-3 and **11-4** reproduce board game-style play, letting you create virtual cards for a dwarf and an elf, name your pieces, and have them fight. The outcome of the fight will impact one of the character's *body points*, which represent the character's health. Be sure to note how OOP can allow you to easily create many identical objects—in this case, dwarves or elves—by “stamping” them out of the predefined template, called a *class*.

```
❶ >>> import random
❷ >>> class Dwarf(object):
    ❸ def __init__(self, name):
        ❹ self.name = name
        self.attack = 3
        self.defend = 4
        self.body = 5
    ❺ def talk(self):
```

```
        print("I'm a blade-man, I'll cut ya!!!")

❹ >>> lenn = Dwarf("Lenn")
    >>> print("Dwarf name = {}".format(lenn.name))
    Dwarf name = Lenn
    >>> print("Lenn's attack strength = {}".format(lenn.attack))
    Lenn's attack strength = 3
    >>>

❺ >>> lenn.talk()
    I'm a blade-man, I'll cut ya!!!
```

Listing 11-3: Imports `random` module, creates a `Dwarf` class, and instantiates a dwarf object

Start by importing `random` to simulate rolling a die ❶; this is how your character will fight. Now define a class for a dwarf character, capitalizing the first letter of the class name, and pass it an `object` argument, which will be the name of your dwarf ❷. A class is a template for creating objects of a certain type. For example, when you create a list or dictionary, you are creating them from a class.

The `Dwarf` class definition is like the card in **Figure 11-2**; it is basically the genetic blueprint for a dwarf. It will assign *attributes*, like strength and vitality, and *methods*, like how the character moves or talks. Attributes are variables scoped to an *instance* of the class, and methods are attributes that also happen to be functions, which are passed a reference to their instance when they run. A class is a data type, and when you create an object of that data type, it is also known as an instance of that class. The process of setting the initial values and behaviors of the instance is called *instantiation*.

Next, define a *constructor* method, also referred to as the *initialization* method. It sets up the initial attribute values for your object ❸. The `__init__()` method is a special built-in method that Python automatically invokes as soon as a new object is created. In this case, you'll pass two arguments: `self` and the `name` of your object.

The `self` parameter is a reference to the instance of this class that is being created, or a reference to the instance a method was invoked on, technically referred to as a *context* instance. If you create a new dwarf and name it “Steve,” `self` will become Steve behind the scenes. For example, `self.attack` becomes “Steve’s attack.” If you create another dwarf named “Sue,” `self` for that object will become “Sue.” This way, the scope of Steve’s health attribute is kept separate from Sue’s.

Next, list some attributes for a dwarf beneath the constructor definition ④. You’ll want a name so you can tell one dwarf from another, as well as the value of key combat characteristics. Notice how this list resembles the card in **Figure 11-2**.

Define a `talk()` method and pass it `self` ⑤. By passing it `self`, you link the method to the object. In more comprehensive games, methods might include behaviors like movement and the ability to disarm traps.

With the class definition complete, create an instance of the `Dwarf` class and assign this object to the local variable `lenn`, the dwarf’s name ⑥. Now, print the name and attack attributes to demonstrate that you have access to them. Finish by invoking the `talk()` method ⑦. This should display a message.

Listing 11-4 creates an elf character, using the same process you used in **Listing 11-3**, and has it fight the dwarf. The elf’s `body` attribute is updated to reflect the outcome of the battle.

```
❶ >>> class Elf(object):
        def __init__(self, name):
            self.name = name
            self.attack = 4
            self.defend = 4
            self.body = 4
    >>> esseden = Elf("Esseden")
    >>> print("Elf name = {}".format(esseden.name))
    Elf name = Esseden
    >>> print("Esseden body value = {}".format(esseden.body))
```

```

    Esseden body value = 4
    >>>
    ❷ >>> lenn_attack_roll = random.randrange(1, lenn.attack + 1)
    >>> print("Lenn attack roll = {}".format(lenn_attack_roll))
    Lenn attack roll = 3
    ❸ >>> esseden_defend_roll = random.randrange(1, esseden.defend + 1)
    >>> print("Esseden defend roll = {}".format(esseden_defend_roll))
    Esseden defend roll = 1
    >>>
    ❹ >>> damage = lenn_attack_roll - esseden_defend_roll
    >>> if damage > 0:
        esseden.body -= damage
    ❺ >>> print("Esseden body value = {}".format(esseden.body))
    Esseden body value = 2

```

Listing 11-4: Creates an `Elf` class, instantiates an `elf` object, simulates a battle, and updates an object attribute

Define an `Elf` class and provide some attributes ❶. Make them slightly different from the dwarf's and well balanced, like an elf. Instantiate an elf named `Esseden` and access his `name` and `body` attributes using `print`.

Have your two characters interact using the roll of a virtual die with a maximum value equal to the character's attack or defend value. Use the `random` module to choose a roll value in a range of 1 to Lenn's `attack` attribute plus 1 ❷, then repeat this process to get Esseden's defense ❸. Calculate the damage to Esseden by subtracting Esseden's roll value from Lenn's roll value ❹, and if the damage is a positive number, subtract it from Esseden's `body` attribute. Use `print()` to confirm the elf's current health ❺.

As you can imagine, building many similar characters and keeping track of their changing attributes could quickly get complicated with procedural programming. OOP provides a modular structure for your program, makes it easy to hide complexity and ownership of scope with encapsulation, permits problem solving in bite-sized chunks, and produces sharable templates that can be modified and used elsewhere.

The Strategy and Pseudocode

Now back to our three-door game. The rules for the game form the bulk of the pseudocode for the program:

```
Initialize game window and show closed doors and instructions
Choose winning door at random
Get player's door choice
Reveal a door that isn't the winning door or the player's choice
Get player's choice to switch doors or not
If player switches:
    Reveal new door
    If winner:
        Record as win for switching
    Otherwise:
        Record as win for staying put
Else if player stays with first choice:
    Reveal chosen door
    If winner:
        Record as win for staying put
    Otherwise:
        Record as win for switching
Display number of wins for each strategy in game window
Reset game and close all doors
```

It's useful to start designing a game by sketching out how the game window should look, complete with instructions, messages, and button types. I doubt you want to see my crude scribbles, so instead check out **Figure 11-3**.

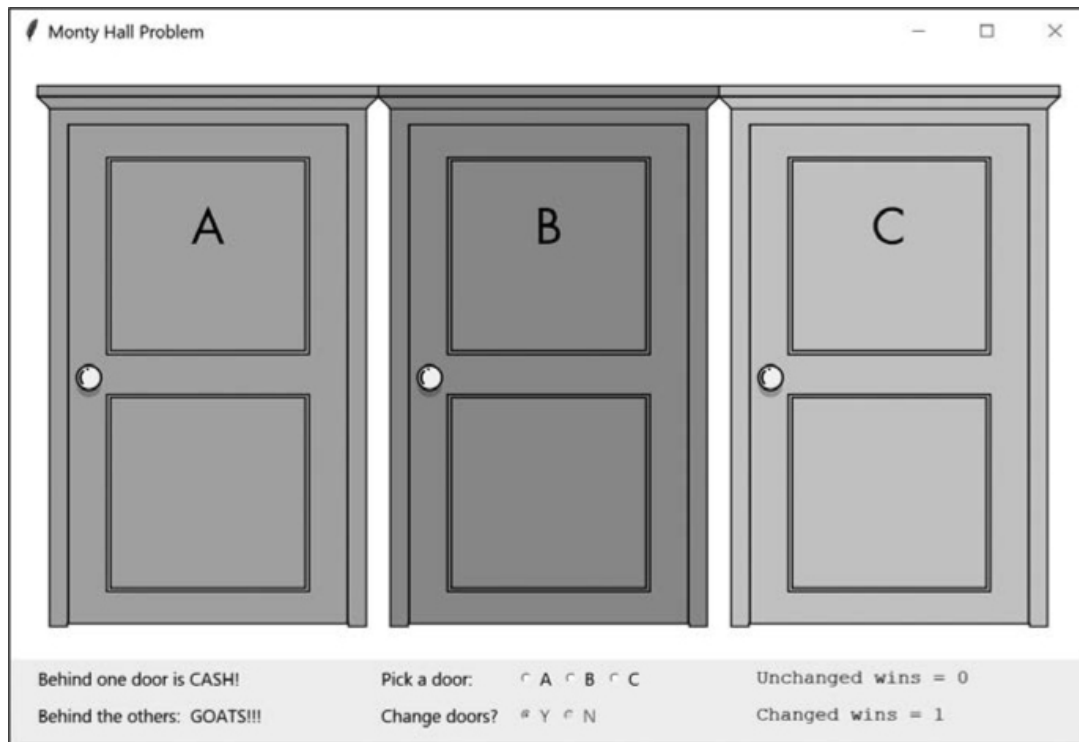


Figure 11-3: View of the game window after the first round of play

This is how the finished game will look after the first round, with the win statistics visible at the far right. Note that the radio buttons for changing doors are grayed out until an initial pick has been made.

Game Assets

Game assets is a fancy term for things that you'll need to build the game. These will consist of a series of images to represent the doors, goats, and prize (**Figure 11-4**).

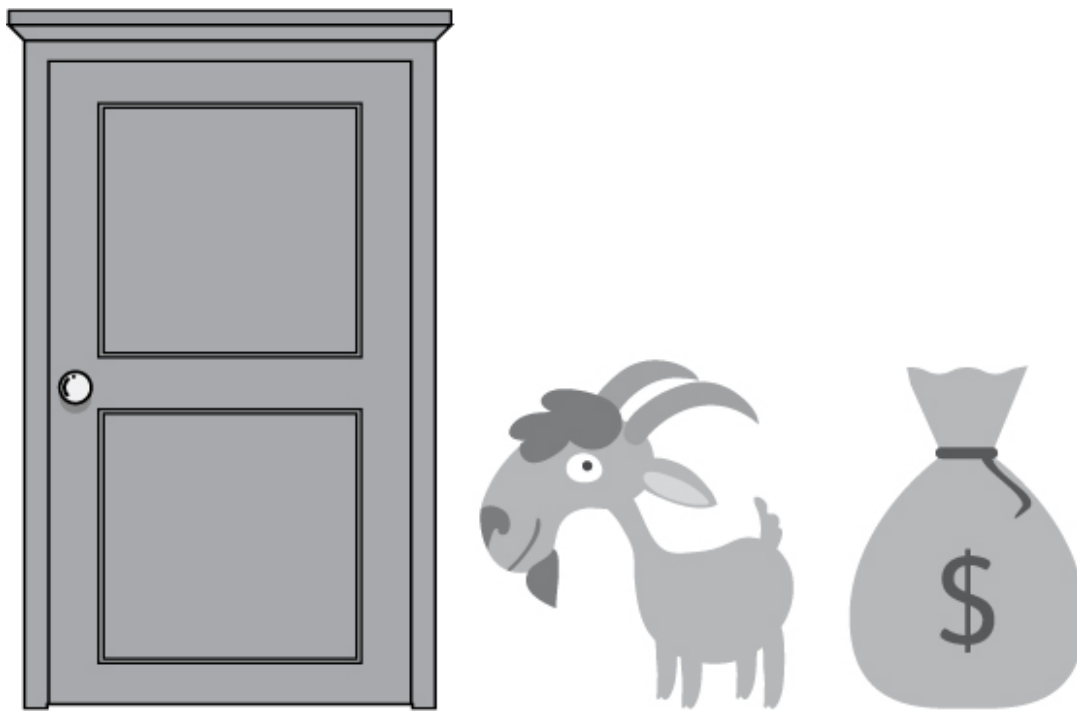


Figure 11-4: Building-block images for the `monty_hall_gui.py` program

I used Microsoft PowerPoint to composite the 3 base images into 10 images that represent all the possible states of the game (**Figure 11-5**). This was a design decision; with extra lines of code, I could have obtained the same results using only the base images.

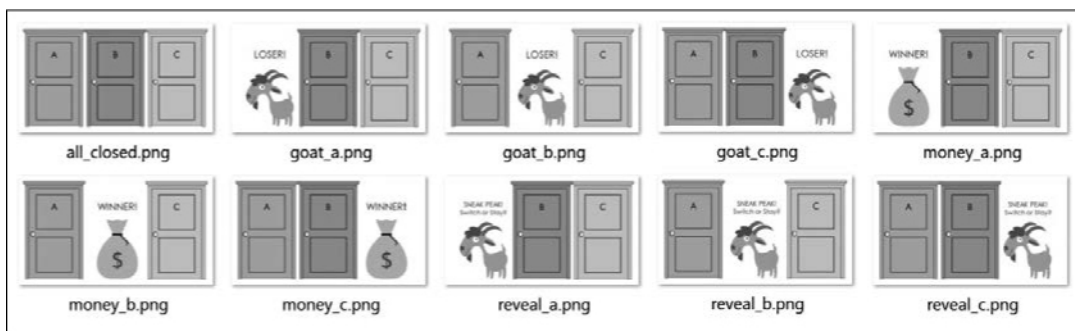


Figure 11-5: Composited images for the `monty_hall_gui.py` program

The Monty Hall Game Code

The `monty_hall_gui.py` program described in this section turns the Monty Hall problem into a fun and educational game. You will also need the 10 game assets shown in **Figure 11-5**. Download them from <https://www.nostarch.com/impracticalpython/> and keep all the files in the same folder.

Importing Modules and Defining the Game Class

Listing 11-5 imports modules and defines the `Game` class and the initialization method, `__init__()`.

monty_hall_gui.py, part 1

```
❶ import random
    import tkinter as tk

❷ class Game(tk.Frame):
    """GUI application for Monty Hall Problem game."""

    ❸ doors = ('a', 'b', 'c')

    ❹ def __init__(self, parent):
        """Initialize the frame."""
        ❺ super(Game, self).__init__(parent) # parent will be the root window
        ❻ self.parent = parent
        self.img_file = 'all_closed.png' # current image of doors
        self.choice = '' # player's door choice
        self.winner = '' # winning door
        self.reveal = '' # revealed goat door
        ❼ self.first_choice_wins = 0 # counter for statistics
        self.pick_change_wins = 0 # counter for statistics
        ❽ self.create_widgets()
```

Listing 11-5: Imports modules and defines the `Game` class and `__init__()` method

Start by importing the `random` and `tkinter` modules ❶. Next, define a class called `Game` ❷. The *ancestor* for this class, shown in parentheses, will be a `tkinter Frame` class. This means that the `Game` class is *derived* from the existing `Frame` “base” class and will conveniently inherit useful methods from it. The `Frame` widget mainly serves as a geometry master for other widgets, helping to group them into complex layouts.

Note that classes have their own docstring conventions, which you can find at <https://www.python.org/dev/peps/pep-0257/>. As stated in

Chapter 1, I will mainly show single-line docstrings in this book for brevity.

Every instance of `Game` will use the same three doors, so you can use a *class attribute* for this ❸. Any variable assigned a value outside of a method becomes a class attribute, much as variables assigned outside of functions in procedural programs become global variables. You don't want this attribute to get changed inadvertently, so make it immutable by using a tuple. Later, you'll make lists from this tuple whenever you want to manipulate the doors.

Now, just as with the earlier dwarf and elf examples, define an initializer method for the game object ❹. A `self` parameter is required, but you'll also need a `parent`, which will be the `root` window that will hold the game.

A base class can also be called a *superclass*, and the `super()` function lets you invoke the method of a superclass to gain access to inherited methods—in this case, from a parent class. First, pass `Game` to `super()`, which means you want to invoke a method of the superclass of `Game`, which is `Frame` ❺. Then pass it `self` as an argument to reference the newly instantiated `Game` object. The `__init__(parent)` part of the statement invokes the initializer method of `Frame` with `parent` (the root window) as the argument. Now attributes in the prebuilt `tkinter Frame` class can be used by your `Game` object. Note that this statement can be simplified to `super().__init__()`.

Next, assign values to a series of instance attributes ❻. It is best to initialize attributes through the `__init__()` method, as it is the first method called after an object is created. This way, these attributes will be immediately available to any other methods in the class. Start by assigning the `parent`, which will be the `root` window, to the instance. Then name an attribute to hold one of the image files (shown in **Figure 11-5**) and assign it the image with all the doors closed, which is what the player will see at the start of each game. Next, name attributes for the player's door choice, the winning door, and the door used to reveal the first goat.

Use a counter to keep track of the number of wins achieved if the player sticks with the first door choice and another to record wins achieved

when the player switches doors ⑦. Finally, call a method that will create the label, button, and text widgets needed to run the game ⑧.

Creating Widgets for Images and Instructions

Listing 11-6 defines the first part of the `create_widgets()` method, used to build the labels, buttons, and text widgets for the game. The first two widgets will be `tkinter` labels used to display the images in **Figure 11-5** and to provide game instructions.

monty_hall_gui.py, part 2

```
① def create_widgets(self):
    """Create label, button, and text widgets for game."""
    # create label to hold image of doors
    ② img = tk.PhotoImage(file='all_closed.png')
    ③ self.photo_lbl = tk.Label(self.parent, image=img,
                               text='', borderwidth=0)
    ④ self.photo_lbl.grid(row=0, column=0, columnspan=10, sticky='W')
    ⑤ self.photo_lbl.image = img

    # create the instruction label
    ⑥ instr_input = [
        ('Behind one door is CASH!', 1, 0, 5, 'W'),
        ('Behind the others: GOATS!!!', 2, 0, 5, 'W'),
        ('Pick a door:', 1, 3, 1, 'E')
    ]
    ⑦ for text, row, column, columnspan, sticky in instr_input:
        instr_lbl = tk.Label(self.parent, text=text)
        instr_lbl.grid(row=row, column=column, columnspan=columnspan,
                       sticky=sticky, ⑧ipadx=30)
```

Listing 11-6: Defines a method to create widgets

Define a method, called `create_widgets()`, that takes `self` as an argument ①. Then assign an attribute to hold an image of the doors ②. Note that you don't have to precede this attribute name with `self`, as it will be used only

locally within the method. The `PhotoImage` class, which takes the image filename as an argument, is used by `tkinter` to display images in canvas, label, text, or button widgets. After this step, you can use the image in a `tkinter` label, so assign a `photo_lbl` variable, pass it the parent and image as arguments, and specify no text and a thin border ❸.

To place the label in the parent window, use the `grid()` method and pass it the first row and first column, let the image span 10 columns, and left-justify it using `w` ❹. This will fill the top part of the window with the image of the closed doors. The `columnspan` option lets the widget span more than one column. The value won't affect the image size but *will* change the number of locations available for placing the instruction text and other widgets below the image. For example, if you set `columnspan=2`, you'll have only two columns available for placing instructions, buttons, and messages.

Finish the photo label by creating a reference to the image object ❺. If you don't do this, the image won't always show up.

According to the `tkinter` documentation, `tkinter` is a layer built over another product (`Tk`), and the interface between the two doesn't handle references to image objects properly. The `Tk` widget holds a reference to the internal object, but `tkinter` doesn't. Python uses a garbage-collector module to automatically reclaim memory from objects no longer needed. When the garbage collector in Python's memory allocator discards the `tkinter` object, `tkinter` tells `Tk` to release the image. But because it's in use, `Tk` can't, so it sets it to transparent instead. Suggestions for solving the problem include using a global variable, using an instance attribute, or as you did here, adding an attribute to the widget instance (`photo_lbl.image = img`). For more information, see <http://effbot.org/tkinterbook/photoimage.htm>.

Finally, add the instruction text as label widgets. The process is to provide a list of parameters and then loop through them to build the widgets. Start with a list of tuples ❻, where each tuple contains the options for making a `Label` object; you can see what each represents in the next statement ❼. As you progress through the `for` loop, create each label in the parent window and assign it some text. Then use the `grid()` method to place the text, based on the information in the list of tuples, into the window.

Use the `ipadx` option with `grid()` ❸. This option refers to internal padding in the x-direction within the label, so you can play with it to tweak the appearance of the text in the window. In this case, you add 30 pixels to the label so that the text will align in a visually pleasing way.

Creating Radio Buttons and Text Widgets

Listing 11-7 continues the definition of the `create_widgets()` method by creating radio button widgets for the three doors. The player makes their initial door choice by selecting the A, B, or C radio button. Their choice is then processed by the `win_reveal()` method, which you'll build later. This method will determine the winning door and reveal a goat.

Another set of radio buttons is created to get the player's choice to switch doors or not. The result will be processed by the `show_final()` method, also defined later. Besides revealing what's behind the player's final door choice, this method will update the win statistics, using `Text` widgets defined at the end of this listing.

monty_hall_gui.py, part 3

```
# create radio buttons for getting initial user choice
❶ self.door_choice = tk.StringVar()
   self.door_choice.set(None)

❷ a = tk.Radiobutton(self.parent, text='A', variable=self.door_choice,
                      value='a', command=self.win_reveal)
   b = tk.Radiobutton(self.parent, text='B', variable=self.door_choice,
                      value='b', command=self.win_reveal)
   c = tk.Radiobutton(self.parent, text='C', variable=self.door_choice,
                      value='c', command=self.win_reveal)

# create widgets for changing door choice
❸ self.change_door = tk.StringVar()
   self.change_door.set(None)

❹ instr_lbl = tk.Label(self.parent, text='Change doors?')
```

```

        instr_lbl.grid(row=2, column=3, columnspan=1, sticky='E')

        ❸ self.yes = tk.Radiobutton(self.parent, state='disabled', text='Y',
                                   variable=self.change_door, value='y',
                                   command=self.show_final)

        self.no = tk.Radiobutton(self.parent, state='disabled', text='N',
                                   variable=self.change_door, value='n',
                                   command=self.show_final)

        # create text widgets for win statistics
        ❹ defaultbg = self.parent.cget('bg')
        ❺ self.unchanged_wins_txt = tk.Text(self.parent, width=20,
                                             height=1, wrap=tk.WORD,
                                             bg=defaultbg, fg='black',
                                             borderwidth=0)

        self.changed_wins_txt = tk.Text(self.parent, width=20,
                                             height=1, wrap=tk.WORD, bg=defaultbg,
                                             fg='black', borderwidth=0)

```

Listing 11-7: Builds radio buttons and text widgets for the `create_widgets()` method

Start by creating radio buttons for doors A, B, and C. When a user interacts with a `tkinter` widget, the result is an *event*. You can use variables to track these events, such as when the player selects a door by pressing a radio button. For widget-specific variables, `tkinter` has a variable class. Use the *string* variable class, `StringVar`, and assign it to a variable named `door_choice` ❶. Immediately use the `set()` method to assign the variable a value of `None`.

Next, set up the button widgets for the three doors ❷. The player will click on one of these for their first door pick. Use the `Radiobutton` class and pass it the parent window, the text to display, the `door_choice` variable you just assigned, a value equal to the door name, and a command. The command calls the `win_reveal()` method, which you'll define shortly. Note that you don't include the parentheses after the method name.

Repeat this process for buttons B and C. This is mainly a cut-and-paste exercise, because all you need to change are the door designations.

Now, build the radio buttons for switching doors. Start by making another string variable, as you did for the initial door choice ❸. This will hold either `y` or `n`, depending on which radio button is selected.

Build an instruction label using the `Label` class ❹. Then build the `self.yes` radio button ❺. Use the `Radiobutton` class, pass it the parent window, and set its state to `disabled`. This way, the window will initialize with the yes/no buttons grayed out, so the player can't jump the gun and try to change a door before first choosing one. The text parameter is the button name; use an abbreviated Y for yes. Set the widget's variable argument to the `change_door` variable, set its value to `y`, and call the `show_final()` function. Repeat the process for the no button.

The last widgets you'll need are `Text` widgets to show the counts for switching doors versus staying put. Use the `Text` class to display the statistics and set the text box color to match the parent window. To do this, use `cget()` to get the background (`bg`) color of `parent` and then assign it to a variable ❻. The `cget()` method returns the current value for a `tkinter` option as a string.

Create a text object to display the wins for sticking with the first choice ❼. You need to pass the widget the parent window, a width and height, how to wrap text if it extends beyond a row, a background color, a foreground color—the text color—and a border width for the text box. Note that you don't include any actual text; this will be added later by the `show_final()` method.

Finish with another text widget to display the number of wins attributed to switching doors.

Arranging the Widgets

Listing 11-8 completes the `create_widgets()` method by using the `tkinter` `Grid` geometry manager to position the remaining nongridDED widgets in the game window.

```
# place the widgets in the frame
❶ a.grid(row=1, column=4, sticky='W', padx=20)
  b.grid(row=1, column=4, sticky='N', padx=20)
  c.grid(row=1, column=4, sticky='E', padx=20)
  self.yes.grid(row=2, column=4, sticky='W', padx=20)
  self.no.grid(row=2, column=4, sticky='N', padx=20)
❷ self.unchanged_wins_txt.grid(row=1, column=5, columnspan=5)
  self.changed_wins_txt.grid(row=2, column=5, columnspan=5)
```

Listing 11-8: Calls the `grid()` method on the widgets to position them in the frame

Use `grid()` to position the door buttons in the parent window ❶. Group the three door buttons together in the same row and column and separate them using the `sticky` justification: `W` means left, `N` is center, and `E` is right. Use `padx` to tweak the positions laterally. Repeat this process for the remaining buttons, then position the win statistics text widgets and allow them to span the five columns on the right side of the window ❷.

Updating the Door Image

You'll need to open and close doors throughout the game, so **Listing 11-9** defines a helper method to update the door image as appropriate. Note that, with OOP, you don't need to pass a filename to the method as an argument. All the methods for an object have direct access to attributes that begin with `self`.

```
❶ def update_image(self):
    """Update current doors image."""
    ❷ img = tk.PhotoImage(file=self.img_file)
```

```
③ self.photo_lbl.configure(image=img)
④ self.photo_lbl.image = img
```

Listing 11-9: Defines a method to update the current door image

Define a function, called `update_image()`, that takes `self` as an argument ❶. Then use the `PhotoImage` class as you did in **Listing 11-6** ❷. The filename, `self.img_file`, will be updated in other methods.

Because you’ve already created the label that holds the door image, use the `configure()` method to change the label—in this case, by loading a new image ❸. You can use either `configure()` or `config()`. Finish by assigning the image to a widget attribute to fend off garbage collection ❹, as described for **Listing 11-6**.

Selecting the Winning Door and Revealing a Goat

Listing 11-10 defines a method that selects the winning door and the reveal door and then opens and closes the reveal door. It also activates the yes/no buttons, which are grayed out until the player makes their first door choice.

monty_hall_gui.py, part 6

```
❶ def win_reveal(self):
    """Randomly pick winner and reveal unchosen door with goat."""
    ❷ door_list = list(self.doors)
    ❸ self.choice = self.door_choice.get()
    self.winner = random.choice(door_list)

    ❹ door_list.remove(self.winner)

    ❺ if self.choice in door_list:
        door_list.remove(self.choice)
        self.reveal = door_list[0]
    else:
        self.reveal = random.choice(door_list)
```

```

        ❹ self.img_file = ('reveal_{}.png'.format(self.reveal))
            self.update_image()

            # turn on and clear yes/no buttons
        ❺ self.yes.config(state='normal')
            self.no.config(state='normal')
            self.change_door.set(None)

            # close doors 2 seconds after opening
        ❻ self.img_file = 'all_closed.png'
            self.parent.after(2000, self.update_image)

```

Listing 11-10: Defines a method to randomly select the winning door and reveal door

Define a method, called `win_reveal()`, that takes `self` as an argument ❶. Immediately make a list of the doors from the class attribute `doors` ❷. You'll alter this list based on the player's first door choice and then the winning door, picked at random by the program.

Now, assign a `self.choice` attribute to the `self.door_choice` string variable, accessed with the `get()` method ❸. The value of this attribute was determined by the door radio button that the user clicked as their first choice. Next, choose the winning door, at random, from the door list.

Remove the winning door from the door list ❹. Then use a conditional to see whether the player's choice is still in the door list; if it is, remove it so that it can't be revealed ❺. This will leave only one door in the list, so assign it to the `self.reveal` attribute.

If the player picked the winning door, there are two doors left in the list, so randomly choose one of them and assign it to `self.reveal`. Update the `self.img_file` attribute for this door ❻, then call the method that updates the photo label to show the new image. **Figure 11-6** is an example of the reveal image for door B.

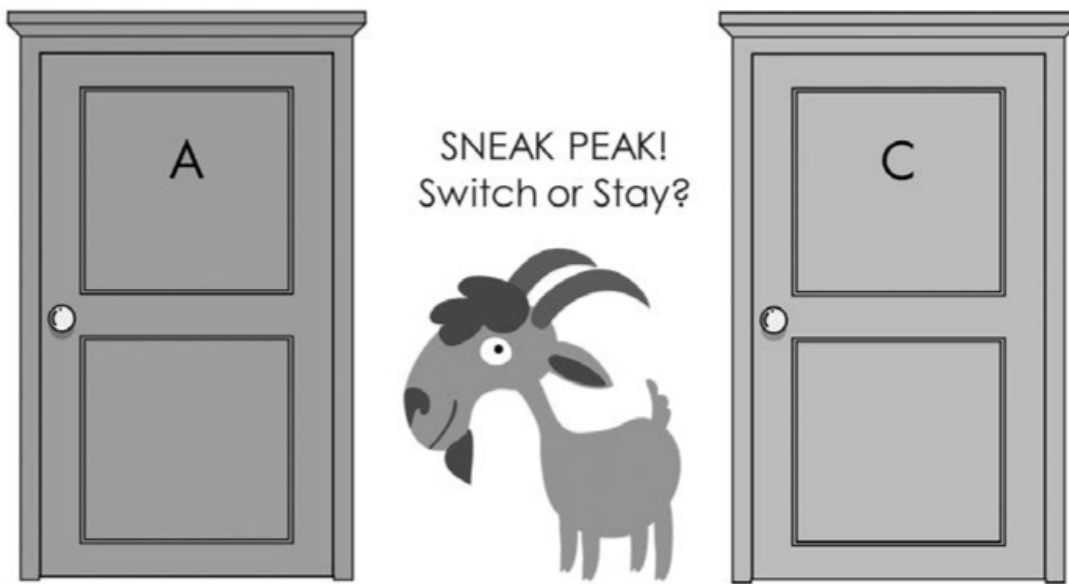


Figure 11-6: The reveal image for Door B

Next, set the state of the yes and no buttons to `normal` ⑦. After this, they will no longer be grayed out. End the method by changing the image file to `all_closed.png` and calling the `self.update_image()` method on the `parent` window after 2,000 milliseconds have elapsed ⑧. This will ensure the doors stay open no longer than 2 seconds.

Revealing the Player's Final Choice

Listing 11-11 defines the first part of a function that takes the player's final door choice and reveals what's behind it. The function will also keep track of the number of wins for switching doors or staying put.

monty_hall_gui.py, part 7

```
① def show_final(self):  
    """Reveal image behind user's final door choice & count wins."""  
    ② door_list = list(self.doors)  
  
    ③ switch_doors = self.change_door.get()  
  
    ④ if switch_doors == 'y':  
        door_list.remove(self.choice)  
        door_list.remove(self.reveal)  
        ⑤ new_pick = door_list[0]
```

```

        ⑥ if new_pick == self.winner:
            self.img_file = 'money_{}.png'.format(new_pick)
            self.pick_change_wins += 1
        else:
            self.img_file = 'goat_{}.png'.format(new_pick)
            self.first_choice_wins += 1
    ⑦ elif switch_doors == 'n':
        ⑧ if self.choice == self.winner:
            self.img_file = 'money_{}.png'.format(self.choice)
            self.first_choice_wins += 1
        else:
            self.img_file = 'goat_{}.png'.format(self.choice)
            self.pick_change_wins += 1

    # update door image
    ⑨ self.update_image()

```

Listing 11-11: Defines a method to reveal the player's final choice and update win lists

Define a method, called `show_final()`, that takes—you guessed it—`self` as an argument ❶. Make a new copy of the door list ❷, then get the `self.change_doors` variable and assign it to an attribute named `switch_doors` ❸. This variable will hold either a 'y' or an 'n', depending on which radio button the player clicked.

If the player chose to switch doors ❹, remove their first choice and the revealed door from the list and assign a `new_pick` attribute to the remaining door ❺. If this new pick is the winning door ❻, reference the proper image and advance the `self.pick_change_wins` counter. Otherwise, set the image to a goat and advance the `self.first_choice_wins` counter.

If the player decides to not change doors ❼ and if their first choice was the winning door ❽, reveal the money bag and advance the `self.first_choice_wins` counter. Otherwise, show a goat and advance the `self.pick_change_wins` counter.

Finish by calling the `update_image()` method to update the image ❹. Again, you don't need to pass it the name of the new image file, as it can access the `self.img_file` attribute that you changed in the preceding code.

Displaying Statistics

Listing 11-12 completes the `show_final()` method by updating the game window for the number of wins statistics, disabling the yes/no buttons, and closing all the doors.

monty_hall_gui.py, part 8

```
        # update displayed statistics
        ❶ self.unchanged_wins_txt.delete(1.0, 'end')
        ❷ self.unchanged_wins_txt.insert(1.0, 'Unchanged wins = {:d}'
                                         .format(self.first_choice_wins))

        self.changed_wins_txt.delete(1.0, 'end')
        self.changed_wins_txt.insert(1.0, 'Changed wins = {:d}'
                                         .format(self.pick_change_wins))

        # turn off yes/no buttons and clear door choice buttons
        ❸ self.yes.config(state='disabled')
        self.no.config(state='disabled')
        ❹ self.door_choice.set(None)

        ❺ # close doors 2 seconds after opening
        self.img_file = 'all_closed.png'
        self.parent.after(2000, self.update_image)
```

Listing 11-12: Displays win statistics, disables the yes/no buttons, and closes all the doors

Start by deleting any text in the `self.unchanged_wins_txt` text widget ❶. Begin deleting at a text index of `1.0`. The format is `line.column`, so you are specifying the first line and first column of the text widget (line numbering starts at 1, column numbering at 0). Finish with `'end'`, which will ensure that all the text after the starting index is deleted.

Next, use the `insert()` method to add the `self.first_choice_wins` attribute value, along with some descriptive text, to the text widget ❷. Begin inserting at text index 1.0.

Repeat this process for the `self.changed_wins_txt` text widget, and then disable the yes/no buttons by setting their `config` state to 'disabled' ❸. Set the `self.door_choice` string variable back to `None`, and you're ready to start a new game ❹.

End the method by closing the doors, as you did in **Listing 11-10** ❺.

Setting Up the Root Window and Running the Event Loop

Listing 11-13 completes the *monty_hall_gui.py* program by setting up the `tkinter` root window, instantiating the game object, and running `mainloop()`. Alternatively, this code could be encapsulated in a `main()` function.

monty_hall_gui.py, part 9

```
# set up root window & run event loop
❶ root = tk.Tk()
❷ root.title('Monty Hall Problem')
❸ root.geometry('1280x820') # pics are 1280 x 720
❹ game = Game(root)
    root.mainloop()
```

Listing 11-13: Sets up the root window, creates a game object, and runs `mainloop()`

The `Tk` class is instantiated without arguments ❶. This creates a top-level `tkinter` widget, which will be the main window of the game application. Assign it to a variable named `root`.

Give the window a title ❷ and a size in pixels ❸. Note that the size of the images influences the geometry so that they fit attractively in the window, with ample room below for instructions and messages.

Now, create the game ④. Pass it the `root` window, which will be the *master* that will contain the game. This results in a new game being placed inside the `root` window.

Finish by invoking the `mainloop()` method on `root`, which keeps the window open and waiting to handle events.

Summary

In this chapter, you used a simple Monte Carlo simulation to confirm that switching doors is the best strategy for the Monty Hall problem. You then used `tkinter` to build a fun interface to let schoolchildren test this conclusion manually, game by game. Best of all, you learned how to use object-oriented programming to build interactive widgets that respond to user input.

Further Reading

Useful `tkinter` references can be found in “**Further Reading**” on **page 212**.

You can find a summary of the 1990 Monty Hall problem controversy online at **<http://marilynvossavant.com/game-show-problem/>**.

Practice Project: The Birthday Paradox

How many people need to be in a room for there to be a 50/50 chance that two of them share the same birth month and day? According to the *birthday paradox*, not that many! As with the Monty Hall problem, the outcome is counterintuitive.

Use MCS to determine how many people it takes to reach the 50 percent mark. Have the program print out the number of people and the probability for a range of room occupants. If you find yourself looking up how to format dates, stop and simplify! You can find a solution, *birthday_paradox_practice.py*, in the appendix or online at **<https://www.nostarch.com/impracticalpython/>**.

